

# Effective Practices for Agentic LLM Workflows in Numerical Relativity

Zachariah Etienne

U of Idaho Physics

October 7, 2026



# KEHpler: rotating neutron-star initial data

- **All-in-one, lightweight rotating neutron-star solver using the KEH approach.**
  - Komatsu–Eriguchi–Hachisu (1989)
- **Broader rotation laws, flexible numerics, integrated validation**
  - Modified j-constant and Uryū rotation laws
  - Configurable grids, Anderson acceleration, and cross-resolution warm starts
  - Built-in Hamiltonian/momentum constraint diagnostics + physical diagnostics

# KEHpler: rotating neutron-star initial data

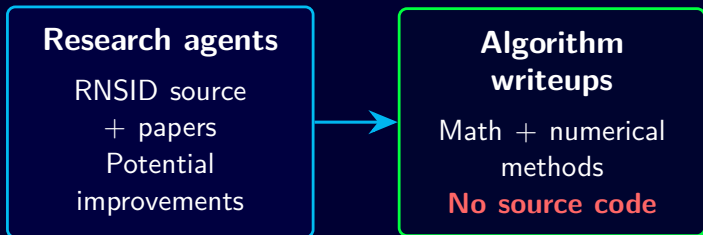
- **All-in-one, lightweight rotating neutron-star solver using the KEH approach.**
  - Komatsu–Eriguchi–Hachisu (1989)
- **Broader rotation laws, flexible numerics, integrated validation**
  - Modified j-constant and Uryū rotation laws
  - Configurable grids, Anderson acceleration, and cross-resolution warm starts
  - Built-in Hamiltonian/momentum constraint diagnostics + physical diagnostics

**Constraint convergence is a required correctness check**

During development, I required Hamiltonian violations to converge toward zero as KEHpler's own grids were refined.

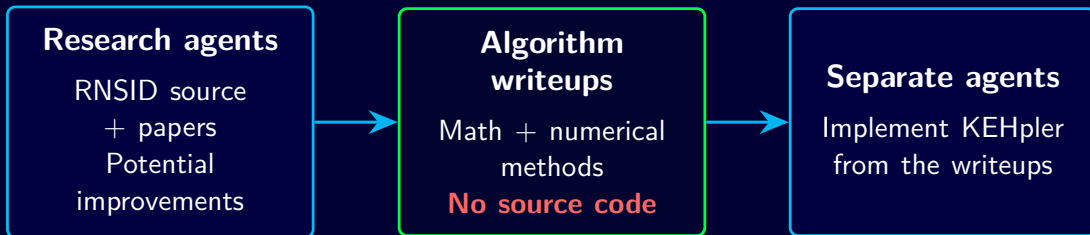
# KEHpler: research, then implementation

**Agent** = a large language model (LLM) that can read and edit files, run commands, and inspect results



# KEHpler: research, then implementation

**Agent = a large language model (LLM) that can read and edit files, run commands, and inspect results**



Implementation agents received the writeups, without the RNSID source.

Context window = limited working space for the current task.

- Long expanded expressions can fill that working space.

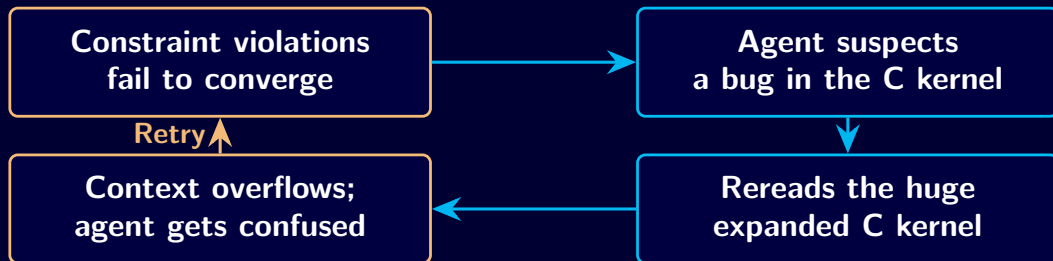
# KEHpler: the debugging loop

- Agents wrote the Hamiltonian constraint directly in C



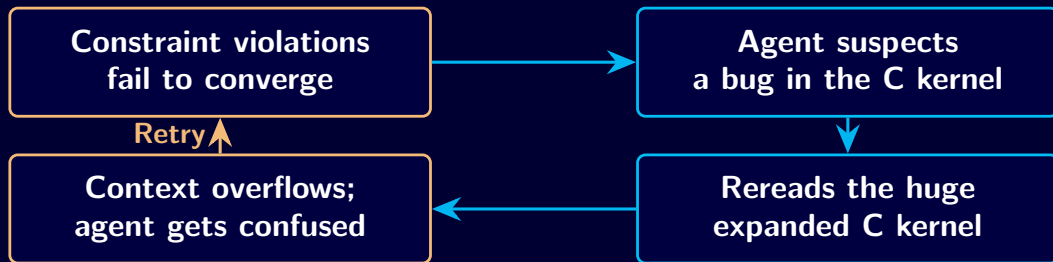
# KEHpler: the debugging loop

- Agents wrote the Hamiltonian constraint directly in C



# KEHpler: the debugging loop

- Agents wrote the Hamiltonian constraint directly in C



- I had agents port KEHpler to NRPpy to keep the math compact

# Einstein notation and component expressions

Einstein notation: repeated indices imply sums

$$\partial_t \gamma_{ij} = -2\alpha K_{ij} + \beta^k \partial_k \gamma_{ij} + \gamma_{ik} \partial_j \beta^k + \gamma_{jk} \partial_i \beta^k$$

# Einstein notation and component expressions

Einstein notation: repeated indices imply sums

$$\partial_t \gamma_{ij} = -2\alpha K_{ij} + \beta^k \partial_k \gamma_{ij} + \gamma_{ik} \partial_j \beta^k + \gamma_{jk} \partial_i \beta^k$$



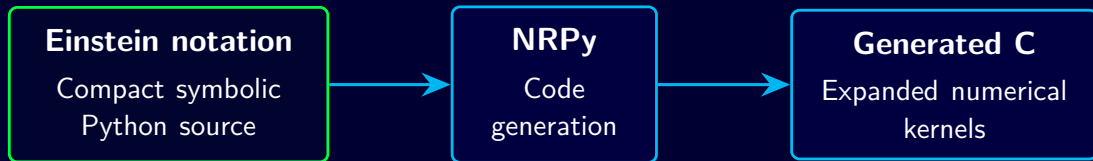
The  $xx$  component, after expanding the sums

$$\begin{aligned} \partial_t \gamma_{xx} = & -2\alpha K_{xx} + \beta^x \partial_x \gamma_{xx} + \beta^y \partial_y \gamma_{xx} + \beta^z \partial_z \gamma_{xx} \\ & + 2\gamma_{xx} \partial_x \beta^x + 2\gamma_{xy} \partial_x \beta^y + 2\gamma_{xz} \partial_x \beta^z \end{aligned}$$

The compact form uses fewer tokens and preserves the structure.

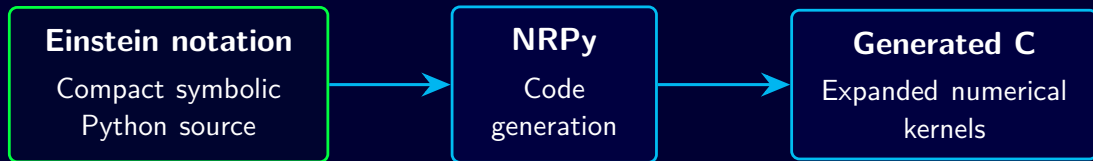
# KEHpler's NRPy port

- I directed the agents to port KEHpler to NRPy



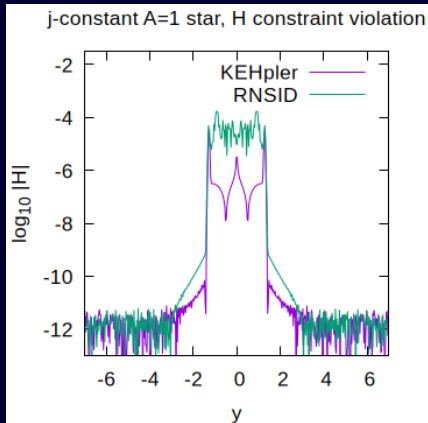
# KEHpler's NRPy port

- I directed the agents to port KEHpler to NRPy



- The equations fit within the agent's context window
- Bugs were fixed quickly; development moved much faster

# Later comparison: constraints on the same NR grid



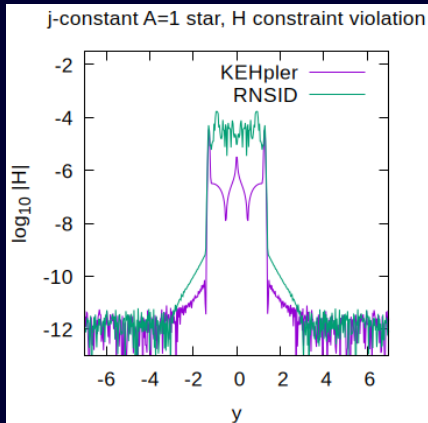
**KEHpler output: ADM metric + matter**

Interpolate onto the NR grid;  
measure the constraints there.

- **j-constant rotation,  $A = 1$**

Same interpolation for both codes

# Later comparison: constraints on the same NR grid



**KEHpler output: ADM metric + matter**

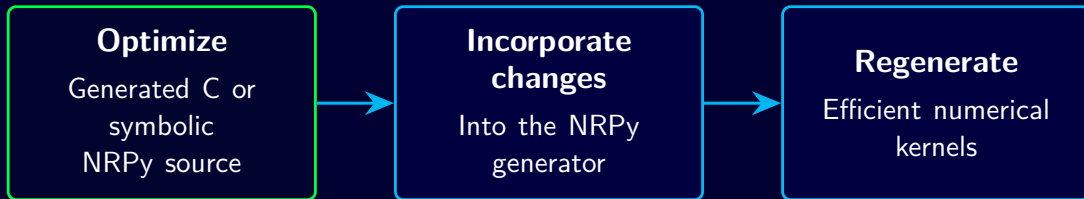
Interpolate onto the NR grid;  
measure the constraints there.

- **j-constant rotation,  $A = 1$**
- **Lower interior Hamiltonian constraint violations**

Same interpolation for both codes

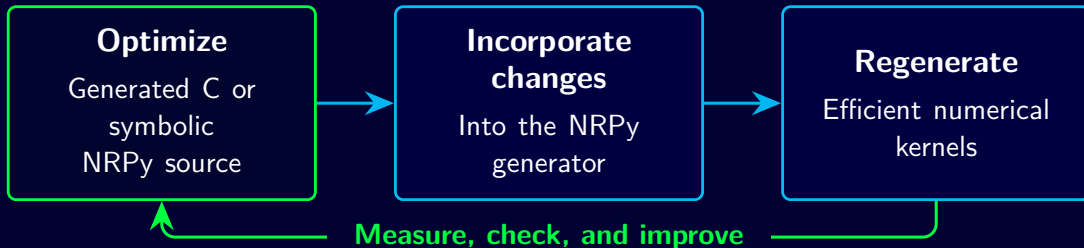
# NRPy kernel optimization

- LLMs can optimize kernels; retain the changes in NRPy's generator.



# NRPy kernel optimization

- LLMs can optimize kernels; retain the changes in NRPy's generator.



Profile the kernels and rerun the numerical checks.

- Record the optimization and its checks in the project wiki (second brain).

# Implementing a new formalism

- Implement the formalism using both its paper and the project's numerical choices.



# Implementing a new formalism

- Implement the formalism using both its paper and the project's numerical choices.



- Which derivation and numerical choices do we already use?

# Implementing a new formalism

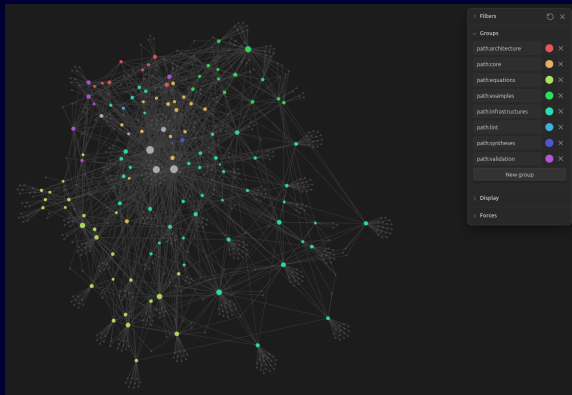
- Implement the formalism using both its paper and the project's numerical choices.



- Which derivation and numerical choices do we already use?
- Which earlier approaches failed, and why?
  - Keep these answers in a project wiki (second brain).

# NRPy's project wiki (second brain)

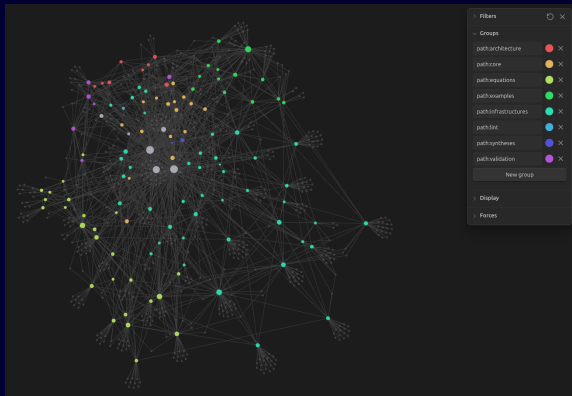
Andrej Karpathy's LLM Wiki: wikis written by and for agents



- Derivations and assumptions

# NRPy's project wiki (second brain)

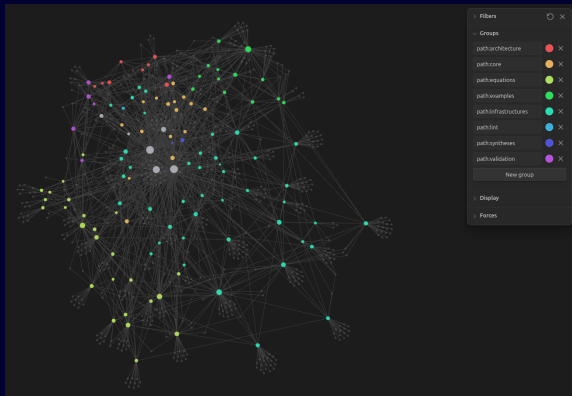
Andrej Karpathy's LLM Wiki: wikis written by and for agents



- Derivations and assumptions
- Numerical approaches that worked or failed

# NRPy's project wiki (second brain)

Andrej Karpathy's LLM Wiki: wikis written by and for agents



- Derivations and assumptions
- Numerical approaches that worked or failed
- Design choices and coding standards

Enforce these coding standards for every proposed patch.

Example: NRPy's fully covariant conformal Z4 (fCCZ4) equations

Has this shift term already been included?

$$(\mathcal{L}_\beta V)^i = \beta^j \partial_j V^i - V^j \partial_j \beta^i$$

Example: NRPy's fully covariant conformal Z4 (fCCZ4) equations

Has this shift term already been included?

$$(\mathcal{L}_\beta V)^i = \beta^j \partial_j V^i - V^j \partial_j \beta^i$$

- The full Lie derivative already includes the highlighted term
- NRPy records why its final expression counts it once

Here  $V^i$  is a vector; the full fCCZ4 equation contains further terms.

# GRHayL test documentation

## General-relativistic hydrodynamics routines



make tests

Compile and link  
test programs

Then

Run the relevant test

Check the numerical result

## General-relativistic hydrodynamics routines



**make tests**

Compile and link  
test programs

**Then**

**Run the relevant test**

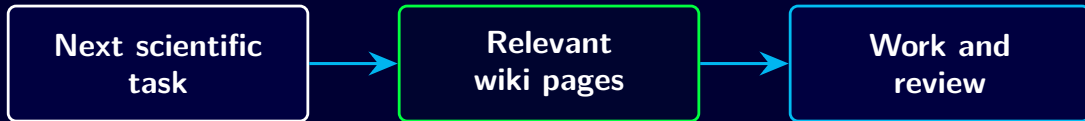
Check the numerical result

“Which test executable checks this routine, what input does it use, and what result does it compare?”

- Save the answer where the next contributor can find it.

# Updating the project wiki

- Save reviewed derivations and implementation choices for reuse.



# Updating the project wiki

- Save reviewed derivations and implementation choices for reuse.



Example note: the full derivation, why we chose this expression, and links to the paper, implementation, and relevant checks.

- Have another agent check the implementation before saving its conclusions.

# Agent-to-agent review

- Agents critique each other's solutions and revise before human review.

## Agent A

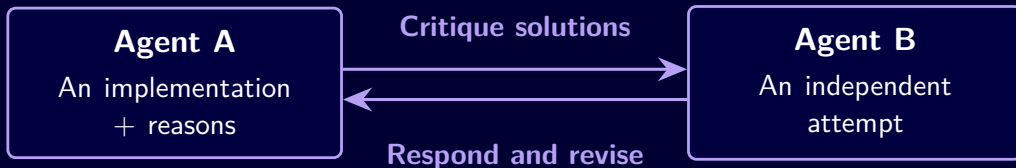
An implementation  
+ reasons

## Agent B

An independent  
attempt

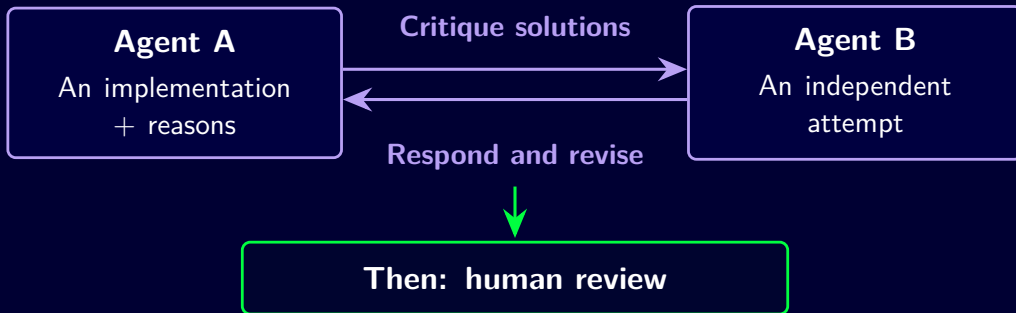
# Agent-to-agent review

- Agents critique each other's solutions and revise before human review.



# Agent-to-agent review

- Agents critique each other's solutions and revise before human review.



- Save the review procedure as reusable instructions.

# Reusable review instructions

**A skill is a reusable set of instructions in a SKILL.md file**

## Example instructions

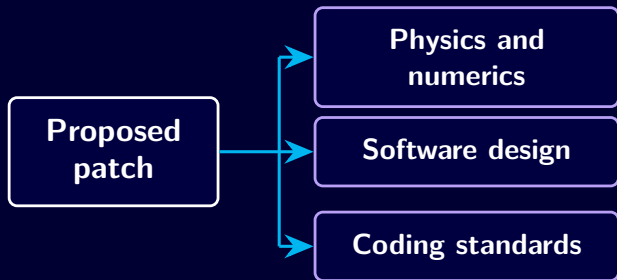
“Work independently from the equations and project notes.  
Compare the proposed implementations. Challenge the assumptions.  
Answer objections with a derivation or a small check, then revise.  
Show the human the revised result and unresolved questions.”

- My automated reviews use a related challenge-and-revise workflow.

# My automated NRPy / GRHayL code reviews

- More proposed patches, limited maintainer time

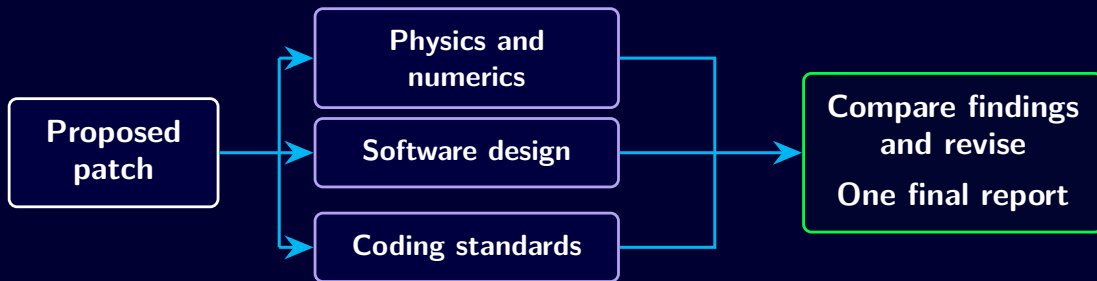
A proposed patch is a code change submitted for review



# My automated NRPy / GRHayL code reviews

- More proposed patches, limited maintainer time

A proposed patch is a code change submitted for review



Each reviewer also challenges its own first findings.

# Code-review report

- A short report I can act on
- The equation or code location that needs attention

# Code-review report

- A short report I can act on
- The equation or code location that needs attention
- An input or argument that shows the problem

# Code-review report

- A short report I can act on
- The equation or code location that needs attention
- An input or argument that shows the problem
- Which checks ran, and what remains unresolved

Less time sorting plausible objections. More time deciding the change.

# Development outcomes

- Faster iteration on scientific software
- Validation of existing human-written codes

# Development outcomes

- Faster iteration on scientific software
- Validation of existing human-written codes
- New physics and improved formulations

# Development outcomes

- Faster iteration on scientific software
- Validation of existing human-written codes
- New physics and improved formulations
- Features and code optimization

My time goes to the scientific decisions and revised results.

# A first agent-assisted review

**Choose a routine or a small proposed patch you already understand**

“Read the project instructions and the pages relevant to this routine.

Explain the equations and trace the callers.

Find one plausible failure case and propose a small check.

Do not edit files yet. Show the sources behind your answer.”

# A first agent-assisted review

**Choose a routine or a small proposed patch you already understand**

“Read the project instructions and the pages relevant to this routine.

Explain the equations and trace the callers.

Find one plausible failure case and propose a small check.

Do not edit files yet. Show the sources behind your answer.”

**Then ask a second agent to critique the reasoning and proposed check.**

# Summary: effective practices for NR

- Efficiency AND Accuracy
- Compact equations for the scientific work
- Project knowledge that survives the conversation
- Agents challenge and revise before human review

